

Отчёт

Ссылка на репозиторий:

<https://github.com/LiDa06/Algorithms/tree/main/SET9>

ID посылок:

- A1m: [375961003](#)

- A1q: [375961085](#)

- A1r: [375961139](#)

- A1rq: [375961291](#)

Теоретические ожидания

Для стандартных алгоритмов сортировки на сравнениях сложность по числу сравнений элементов составляет порядка: $O(n \log n)$

Однако при сортировке строк одно сравнение двух элементов может требовать просмотра нескольких символов. Поэтому реальная стоимость сравнения зависит от длины общего префикса строк. В худшем случае стандартные сортировки имеют сложность порядка: $O(n \log n * L)$ где L — средняя длина сравниваемого префикса.

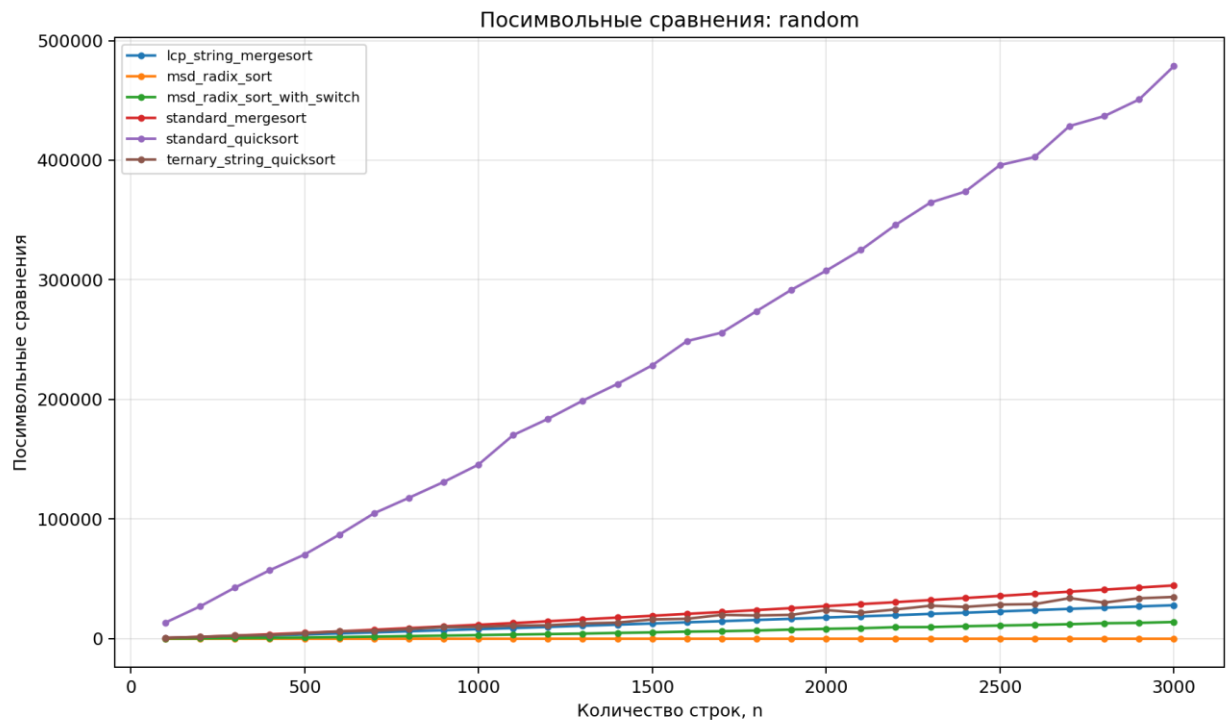
Специализированные алгоритмы строковой сортировки работают иначе. Они стараются не сравнивать одни и те же префиксы повторно:

- *ternary string quicksort* группирует строки по символу на текущей глубине;
- *LCP string mergesort* использует информацию о наибольшем общем префиксе;
- *MSD radix sort* распределяет строки по текущему символу и рекурсивно сортирует корзины.

Поэтому для строк с длинными совпадающими префиксами специализированные алгоритмы должны показывать преимущество по количеству посимвольных сравнений и обращений.

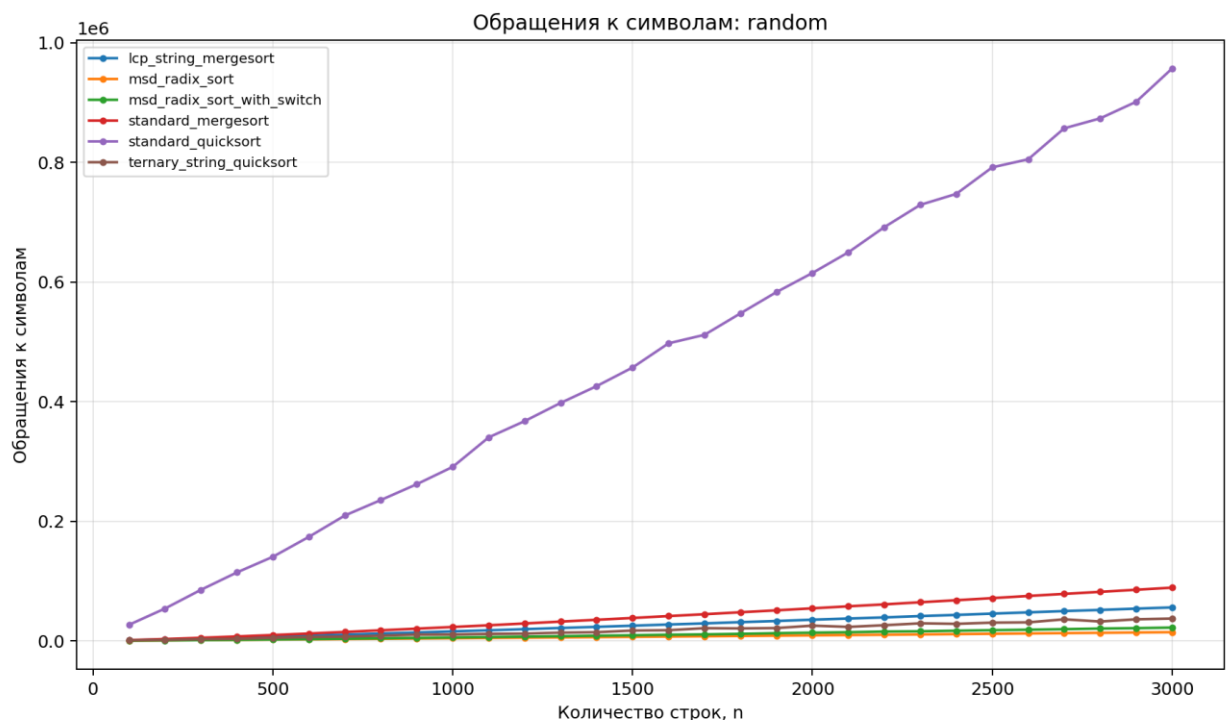
Анализ результатов

Случайные строки



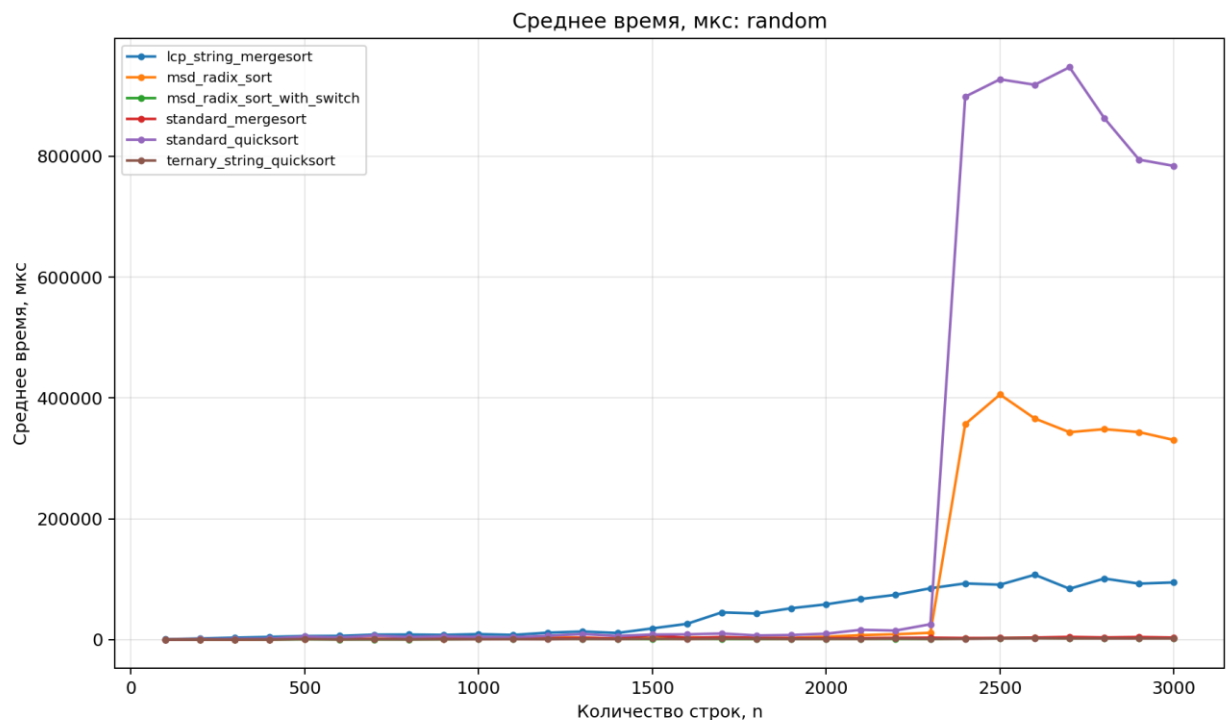
На графиках для видно, что стандартный *quicksort* выполняет наибольшее количество посимвольных сравнений. При $n = 3000$ оно достигает примерно 480 тысяч сравнений, тогда как у специализированных алгоритмов значения существенно ниже.

Стандартный *mergesort* работает стабильнее, чем стандартный *quicksort*, но всё равно выполняет больше посимвольных операций, чем специализированные алгоритмы.



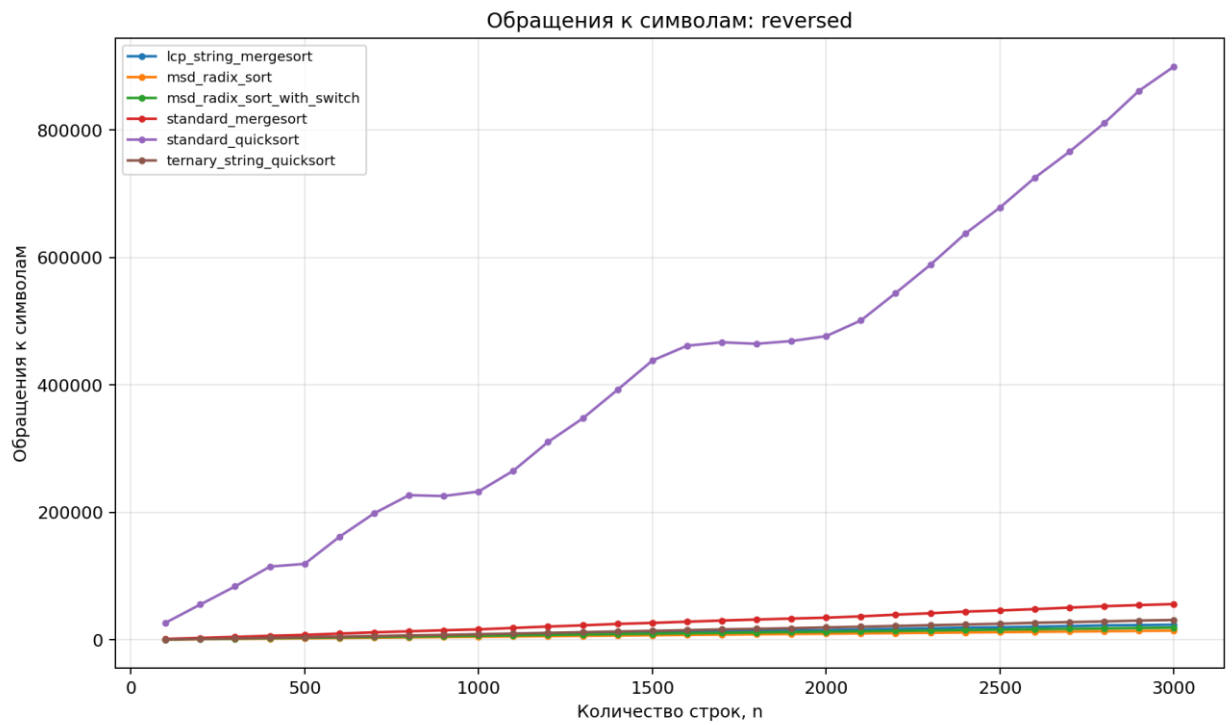
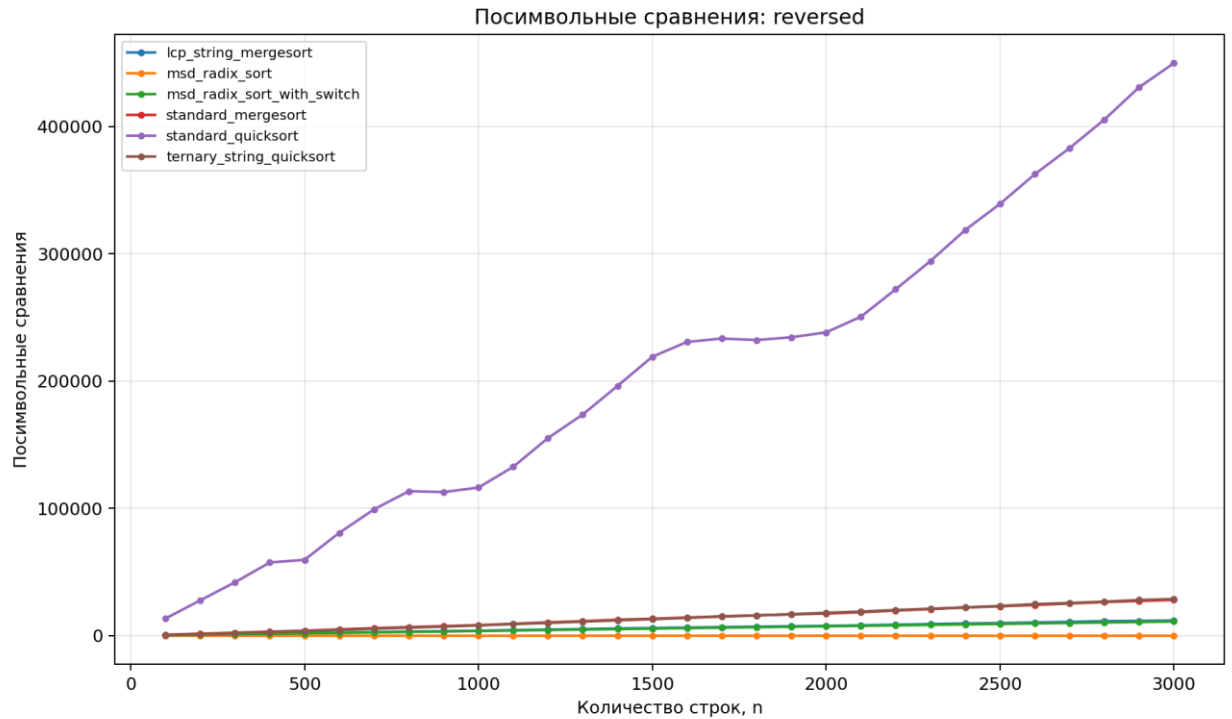
Лучшие результаты по числу обращений к символам показывают *MSD radix*

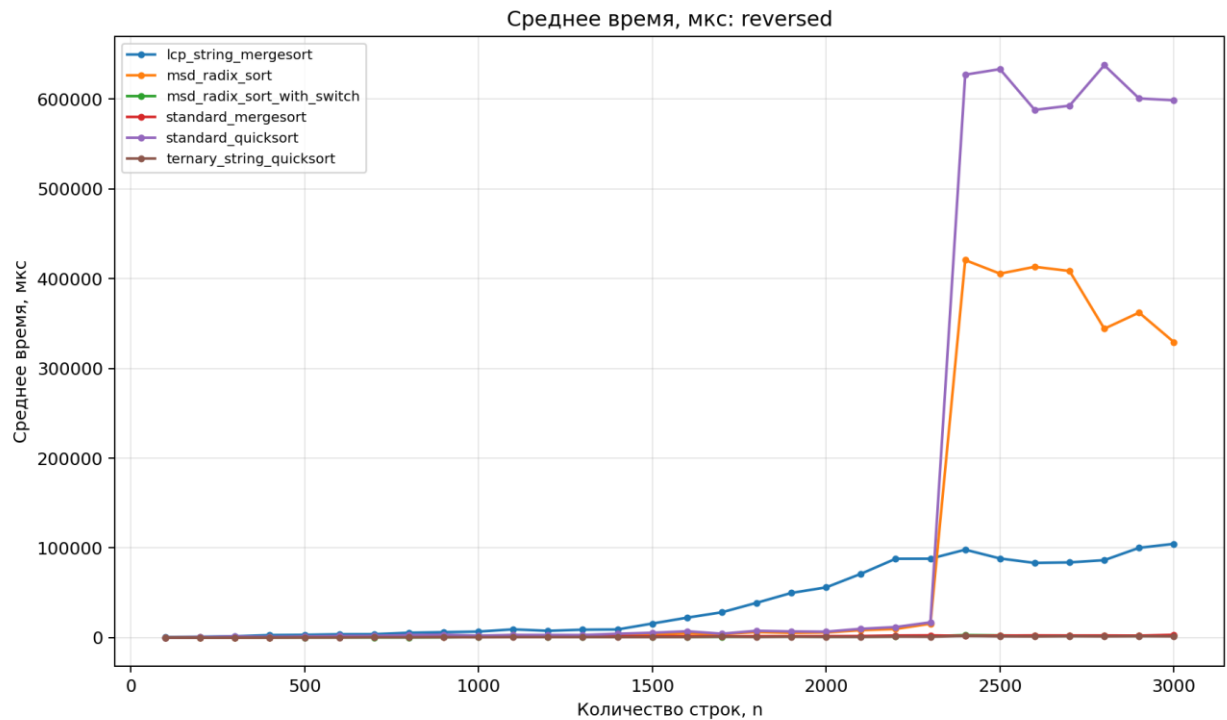
sort и *MSD radix sort with switch*. Это соответствует теории: radix-сортировка не выполняет полные попарные сравнения строк, а обрабатывает символы по позициям.



По времени работы наиболее заметно, что стандартный *quicksort* резко проигрывает при больших n . Также обычный *MSD radix sort* без переключения показывает значительный рост времени на больших размерах, несмотря на малое число сравнений. Это связано с накладными расходами рекурсии и обработки большого числа малых подмассивов. Версия с переключением на *string quicksort* работает заметно стабильнее.

Обратно отсортированные строки



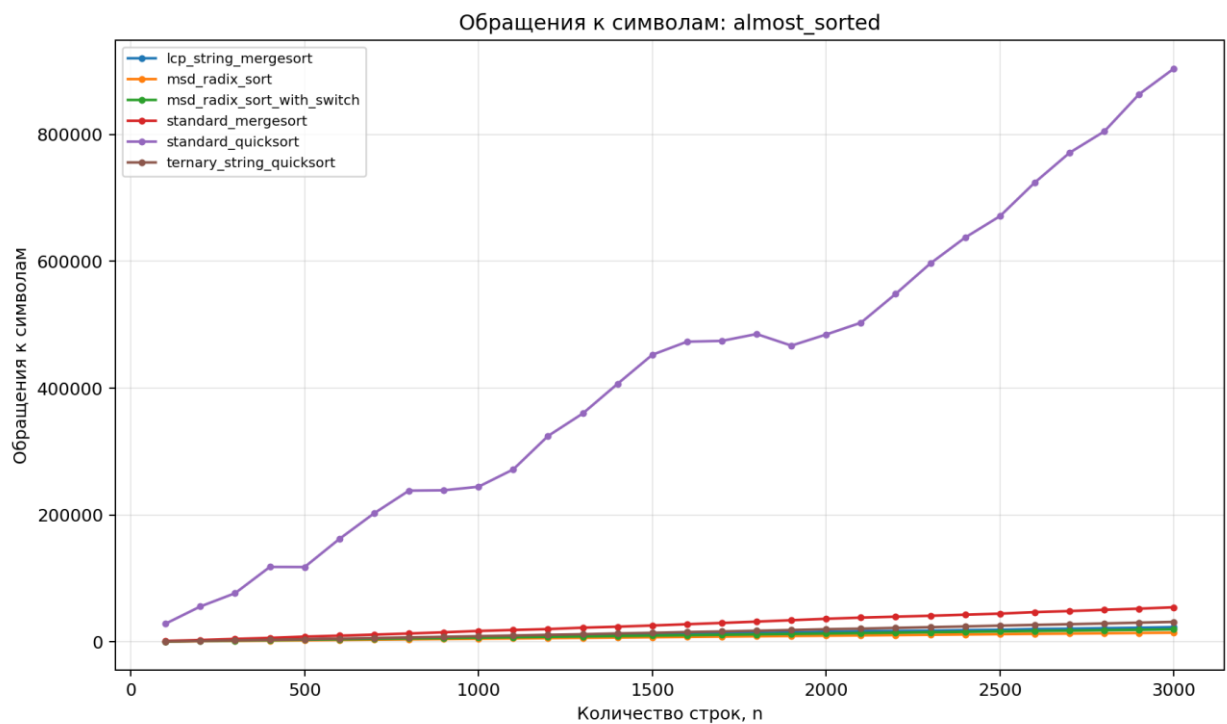
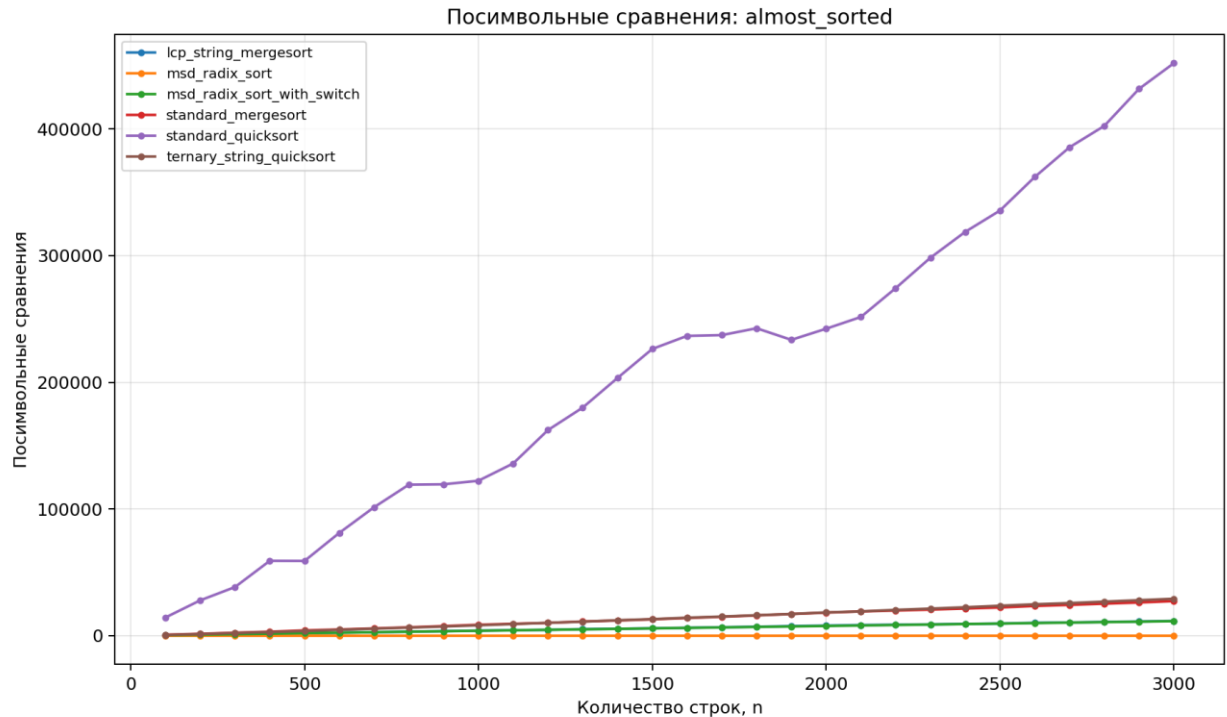


Для набора *reversed* картина похожа на случайные данные. Стандартный *quicksort* снова имеет наибольшее количество посимвольных сравнений и обращений к символам. Это связано с тем, что обычная быстрая сортировка многократно выполняет лексикографические сравнения строк.

Стандартный *mergesort* показывает более предсказуемое поведение, однако уступает специализированным алгоритмам по количеству операций с символами.

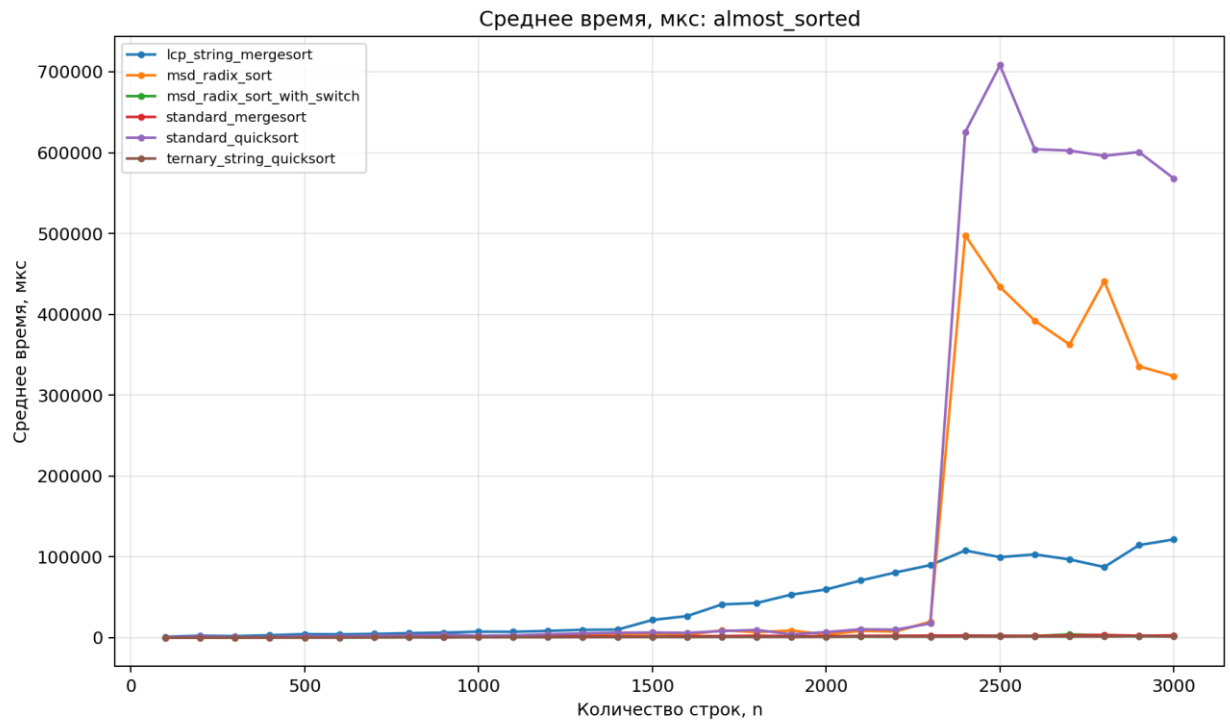
MSD radix sort with switch и *ternary string quicksort* демонстрируют лучшие практические результаты. Переключение на тернарный *quicksort* для малых подмассивов уменьшает накладные расходы radix-сортировки.

Почти отсортированные строки



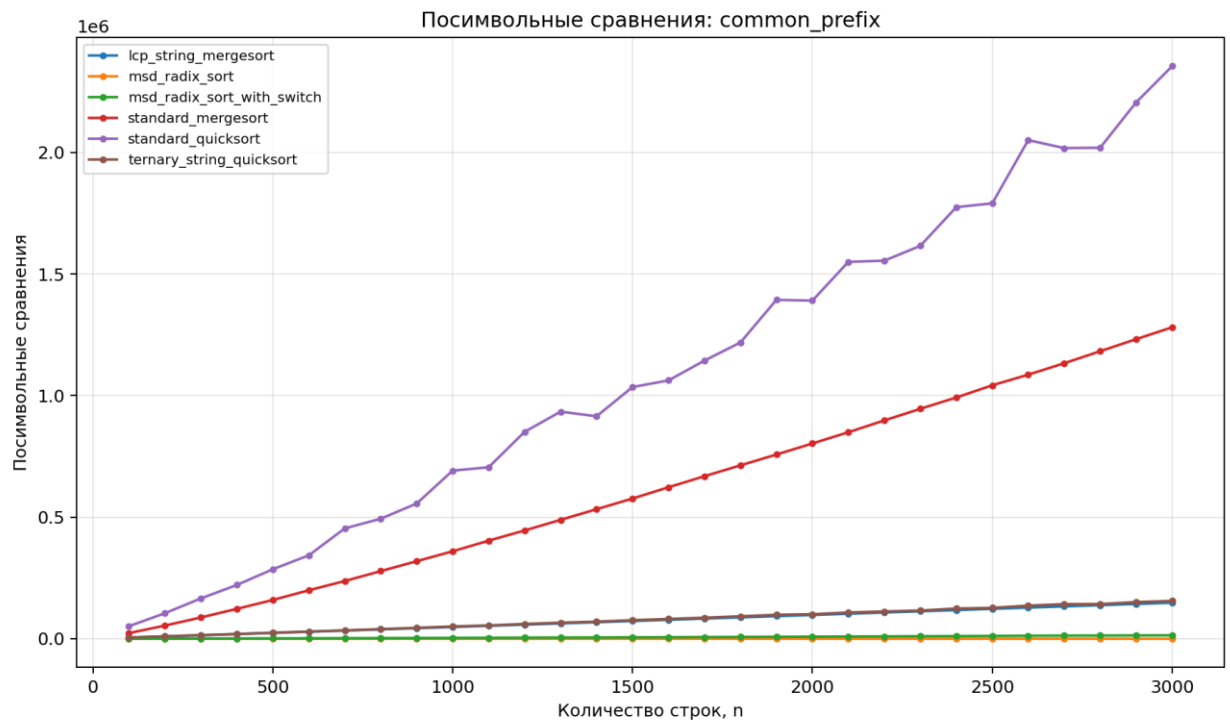
В этом наборе стандартный *quicksort* снова оказался худшим по количеству посимвольных сравнений. Его график растёт значительно быстрее остальных.

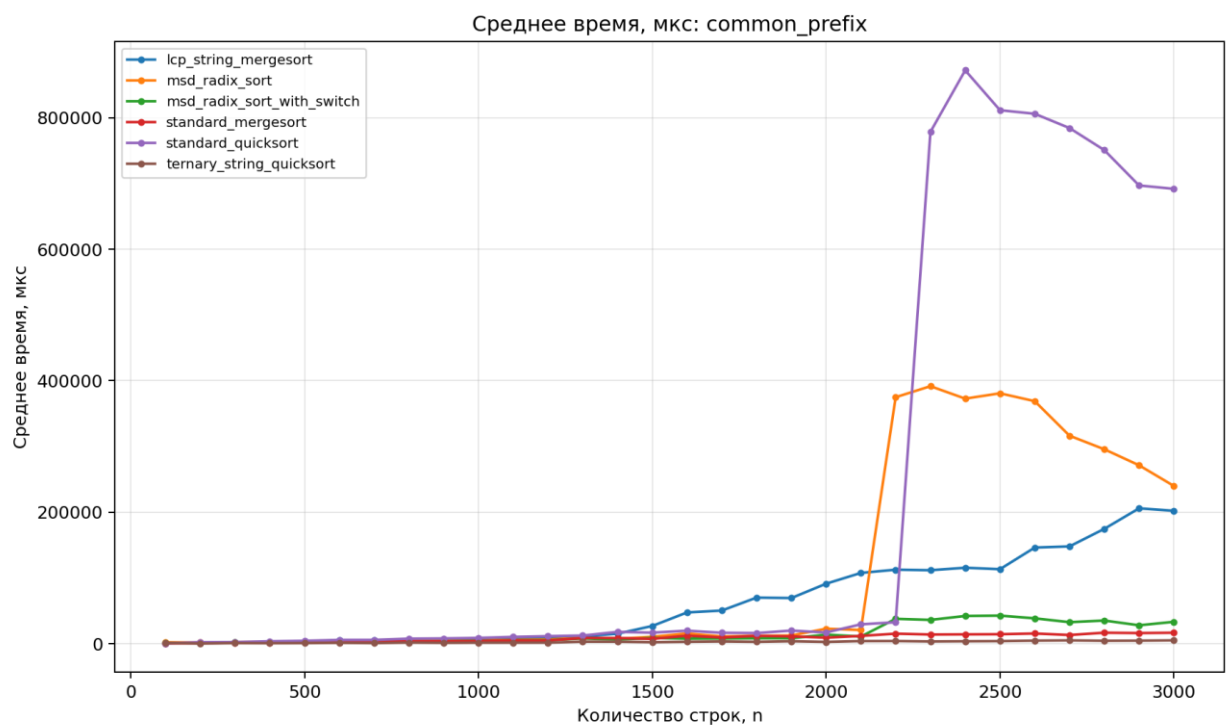
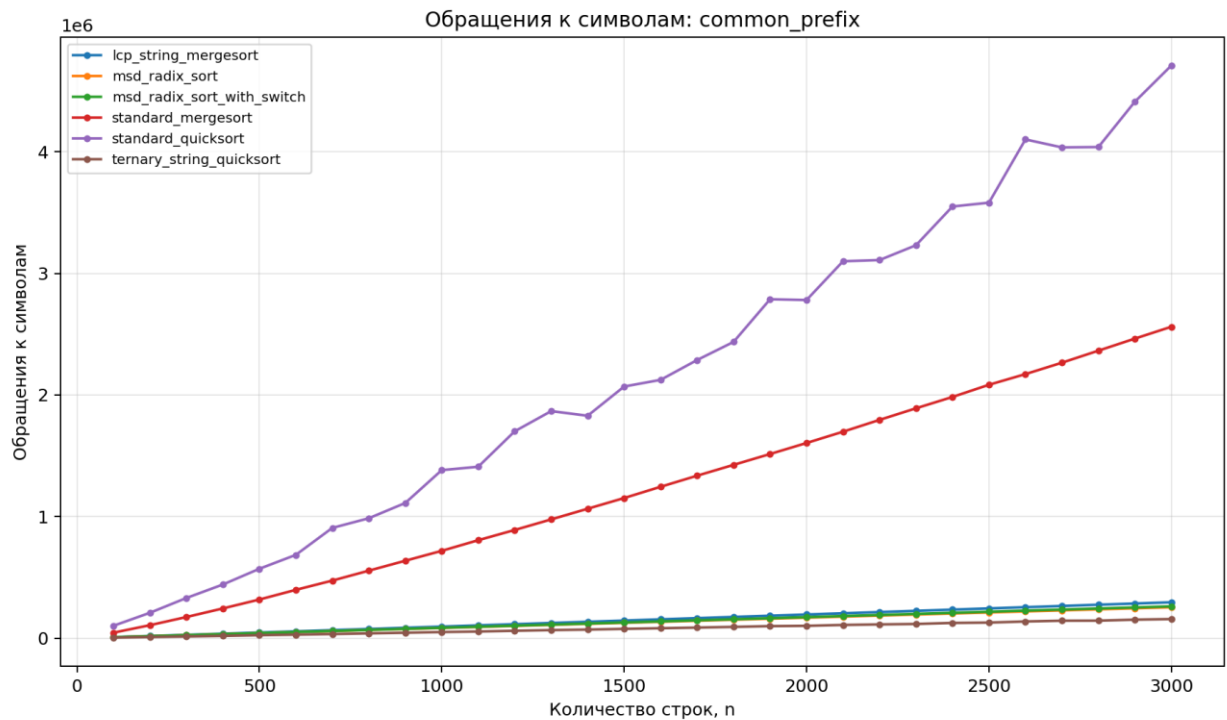
При этом специализированные алгоритмы почти не зависят от того, что массив уже близок к отсортированному состоянию. Они продолжают работать с префиксами и символами строк, а не только с порядком элементов.



ternary string quicksort, *MSD radix sort with switch* и *LCP string mergesort* показывают небольшое число посимвольных операций. Однако по времени *LCP string mergesort* оказывается медленнее, чем можно ожидать только по количеству сравнений. Это объясняется тем, что реализация *LCP-merge* требует дополнительного хранения структур, перемещений строк и работы с временными массивами.

Строки с общим префиксом





Стандартные алгоритмы резко увеличивают количество операций. Особенно сильно растёт число посимвольных сравнений у стандартного *quicksort*: при $n = 3000$ оно превышает 2 миллиона. Стандартный *mergesort* также показывает большой рост — более 1 миллиона сравнений.

Это полностью соответствует теоретическому ожиданию: если строки имеют длинный общий префикс, обычное лексикографическое сравнение вынуждено каждый раз заново проходить одинаковые начальные символы.

Специализированные алгоритмы ведут себя значительно лучше. *ternary string quicksort*, *LCP string mergesort* и *MSD radix sort with switch* используют

информацию о совпадающих префиксах и избегают большого числа повторных сравнений.

Особенно хорошо на этом наборе выглядит *ternary string quicksort*: он имеет малое количество операций и одно из лучших времён работы. *MSD radix sort with switch* также показывает хорошие результаты, тогда как обычный *MSD radix sort* без переключения становится заметно медленнее из-за большого числа рекурсивных вызовов и обработки корзин.

Выводы

Практические результаты подтверждают теоретическое предположение: для сортировки строк специализированные алгоритмы оказываются эффективнее стандартных сортировок сравнением, особенно если строки имеют длинные совпадающие префиксы.